DEPARTMENT OF COMPUTER ENGINEERING

F1 2025 GRAND PRIX RESULTS

Data Analysis and Visualization Using Python

Project Title	F1 2025 Grand Prix Results — Data Analysis & Visualization
Domain	Sports Data Analytics (Formula 1 Motorsport)
Prepared By	Soham Hande & Anshu Navghare
Project Type	Team Project (2 Members)
Class / Year	Second Year Engineering (VSEC)
Academic Year	2025 – 2026
Assignment	Assignment – 12 (Mini Project)
Tools Used	Python · Pandas · NumPy · Matplotlib · Seaborn

PROJECT GITHUB REPOSITORIES

Soham Hande

github.com/handesoham1234-tech/Soham_Hande_19

Anshu Navghare

github.com/maverick8307/anshu_02

Soham Hande **Anshu Navghare**

Scan either QR code to open the corresponding GitHub repository

TABLE OF CONTENTS

1. Title of the Project 3

2. Aim / Objective 4

3. Dataset Description 5

4. Tools and Technologies 7

5. Data Cleaning with Pandas 8

6. Data Analysis with NumPy 10

7. Data Visualization with Matplotlib / Seaborn 13

8. Findings / Observations 18

9. Result / Screenshots 19

10. GitHub Repository Links 22

11. Conclusion 23

12. References 24

This report documents the complete lifecycle of the team mini project — starting from raw dataset inspection, Pandas-based cleaning, NumPy-based statistical analysis, Matplotlib/Seaborn visualisation, and the final interpretation of results. Every figure and table in this document is generated programmatically from the source dataset by the Python script **f1_analysis.py**, which is uploaded to both team GitHub repositories listed in Section 10.

1. Title of the Project

"F1 2025 Grand Prix Results — Data Analysis and Visualization Using Python"

Formula 1 (F1) is the highest class of international single-seater motorsport. The 2025 FIA Formula One World Championship consisted of a record **24 Grand Prix races** held across 21 countries, contested by **21 drivers** representing **10 constructor teams**. Every race produces a rich set of structured records — grid positions, finishing positions, lap counts, time gaps, championship points, fastest-lap data and reliability status.

This team project treats the complete 2025 season as a real-world sports analytics dataset and applies the standard data-science workflow on it: **clean** the data with Pandas, **analyse** it numerically with NumPy, **visualise** the patterns with Matplotlib and Seaborn, and **communicate** the findings through this structured report.

1.1 Problem Statement

Raw motorsport result tables contain missing values for retired drivers, inconsistent country labels, invalid negative gap values and mixed data types. Answering questions such as *“Who won the championship?”, “How strongly does qualifying position influence the race result?”* and *“Which team was the most consistent?”* is impossible until the data is cleaned and summarised. This project solves exactly that problem.

2. Aim / Objective

The aim of this mini project is to choose a real-world sports dataset (the F1 2025 Grand Prix results) and perform an end-to-end data analysis and visualisation study using the Python data-science stack.

2.1 Specific Objectives

- To load and inspect the raw Formula 1 2025 result dataset and understand its structure, data types and quality.
- To clean the dataset using **Pandas** — handle missing values, remove/correct invalid values, fix inconsistent labels, standardise data types and engineer useful new columns.
- To perform numerical analysis using **NumPy** — compute average, minimum, maximum, total, standard deviation, median and correlation coefficients.
- To create at least 3–5 professional visualisations using **Matplotlib** and **Seaborn** (the project delivers ten charts) with proper titles and axis labels.
- To derive championship standings for drivers and constructors, and quantify race wins, podiums, pole positions and fastest laps.
- To interpret every result and document the findings, screenshots, conclusion and references in a structured report.

2.2 Scope of the Project

The study covers all 24 rounds of the 2025 season and every driver entry (479 records). It is restricted to descriptive and diagnostic analytics — it explains *what* happened and *why*, rather than predicting future races (predictive modelling is suggested as future work).

3. Dataset Description

Dataset Name	F1 2025 Grand Prix Results (f1_2025_grand_prix_results.csv)
Domain / Category	Sports — Formula 1 motorsport race results
File Format	Comma-Separated Values (.csv), UTF-8 encoded
Source	Official Formula 1 results (formula1.com) compiled public dataset
Number of Records	479 rows (one row per driver per race entry)
Number of Columns	33 columns in the raw file; 25 retained after cleaning
Races Covered	24 Grand Prix rounds (March – December 2025)
Drivers / Teams	21 drivers across 10 constructor teams
Sprint Weekends	6 (China, Miami, Belgium, USA, São Paulo, Qatar)
Countries Visited	21 countries on 5 continents

The dataset contains detailed information about every race entry: the event (round, Grand Prix name, date, country, circuit, sprint flag), the driver and team (names, nationalities, car number), the qualifying and race outcome (grid position, finish position, status, points, podium/winner flags), lap information (laps completed, completion percentage) and performance timing (gap to winner, fastest lap and its lap number).

3.1 Important Attributes (Data Dictionary)

Attribute	Meaning	Data Type
round / grand_prix	Race number and name of the Grand Prix	int / text
race_date / country	Date of race and host country (21 nations)	date / text
circuit	Name of the racing circuit	text
sprint_weekend	True if the event included a sprint race	boolean
driver / driver_code	Full driver name and 3-letter abbreviation	text
team	Constructor / team the driver raced for	text
grid_position	Starting position after qualifying	integer (1–20)
finish_position	Final classified finishing position	integer (1–20)
result_status	Finished / Lapped / Retired / DSQ / DNS	category

Attribute	Meaning	Data Type
laps_completed	Number of race laps the driver completed	integer
gap_to_winner_seconds	Gap to the race winner in seconds	float
points	Championship points awarded for the entry	float (0–25)
fastest_lap	Driver's fastest lap time (m:ss.mmm)	text
grid_places_gained	Places gained/lost from grid to finish	integer
winner / podium / points_finish	Boolean flags for top-1, top-3, points zone	boolean

3.2 Sample Records (first three rows of raw data)

Round	Grand Prix	Driver	Team	Grid	Finish	Points
1	Australia	Lando Norris	McLaren	1	1	25
1	Australia	Max Verstappen	Red Bull Racing	3	2	18
1	Australia	George Russell	Mercedes	4	3	15

4. Tools and Technologies

The project is built entirely with free, open-source and industry-standard tools. The table below lists each technology and its role in the workflow.

Tool / Technology	Version	Role in the Project
Python	3.x	Core programming language used for the complete analysis pipeline.
Pandas	2.2.x	Loads the CSV, inspects data types, handles missing values, corrects invalid data and groups/aggregates records.
NumPy	2.3.x	Performs numerical computation — mean, min, max, sum, std, median and the correlation matrix (np.corrcoef).
Matplotlib	3.10.x	Draws bar, line, histogram, scatter and pie charts with custom titles, labels and colours.
Seaborn	0.13.x	Builds statistical graphics — heatmap, count plot, box plot — on top of Matplotlib.
Jupyter / Colab	—	Compatible notebook environments; the same code runs unchanged in Jupyter Notebook or Google Colab.
ReportLab	5.x	Generates this final multi-page PDF report programmatically.
PIL / qrcode	—	Renders IDE/terminal-style screenshots and the GitHub QR codes.

4.1 Why this Technology Stack?

- Pandas DataFrames are the de-facto standard for tabular data cleaning in Python and handle missing data elegantly.
- NumPy operates on fast, memory-efficient arrays and provides rigorously tested statistical functions.
- Matplotlib gives full low-level control over every chart element, while Seaborn adds beautiful statistical defaults.
- All libraries are free, cross-platform and reproducible — the entire analysis can be re-run with a single command.

5. Data Cleaning with Pandas

Data cleaning (pre-processing) is the most important stage because the quality of every later result depends on it. The raw file was first copied into a working DataFrame so that the original data was never modified.

5.1 Missing Values Found (before cleaning)

olumn with Missing Value	Missing Count	Percentage	Reason
race_time_or_gap	60	12.5 %	Retired / DSQ / DNS entries have no time
gap_to_winner_seconds	55	11.5 %	Non-classified drivers carry no gap
fastest_lap_rank	22	4.6 %	Driver never set a valid timed lap
fastest_lap	22	4.6 %	No timed lap recorded
fastest_lap_number	22	4.6 %	No timed lap recorded
TOTAL MISSING CELLS	181	—	Across the whole raw dataset

Duplicate records found: 0. Pandas `duplicated()` confirmed that no row was repeated, so no deletion was required.

5.2 Cleaning Actions Performed

#	Cleaning Step (Pandas)
1	Loaded the raw CSV with pd.read_csv() and created a safe working copy using .copy().
2	Stripped accidental leading/trailing whitespace from every text column with str.strip().
3	Corrected inconsistent labels: country value 'UK' was renamed 'United Kingdom' in 20 rows.
4	Converted race_date from plain text to a proper datetime64 type with pd.to_datetime().
5	Filled 55 missing gap_to_winner_seconds values with 0.0 (non-classified drivers have no valid gap).
6	Corrected 5 invalid NEGATIVE gap values belonging to Retired drivers to 0.0 — a time gap can never be negative.
7	Replaced 60 missing race_time_or_gap entries with the explicit label 'Not Classified'.

#	Cleaning Step (Pandas)
8	Filled 22 missing fastest-lap fields with 0 / 'No Timed Lap' and cast rank columns to integer.
9	Engineered a new column fastest_lap_seconds by converting 'm:ss.mmm' lap strings into pure seconds.
10	Selected the 25 analysis-ready columns (dropped 8 redundant URL / text columns) and saved f1_2025_cleaned.csv.

5.3 Pandas Code Used — VSEC Syllabus Concepts

```
# Data handling with Pandas (read_csv, isnull, fillna, replace,
# to_datetime, duplicated, astype, apply, to_csv)
raw = pd.read_csv("f1_2025_grand_prix_results.csv")
print(raw.shape) # (479, 33)
print(raw.isna().sum()) # locate missing values
print(raw.duplicated().sum()) # duplicate-row check -> 0

df = raw.copy()
df["country"] = df["country"].replace({"UK": "United Kingdom"})
df["race_date"] = pd.to_datetime(df["race_date"])
df["gap_to_winner_seconds"] = df["gap_to_winner_seconds"].fillna(0.0)
df.loc[df["gap_to_winner_seconds"] < 0,
"gap_to_winner_seconds"] = 0.0
df["fastest_lap_rank"] = df["fastest_lap_rank"].fillna(0).astype(int)
df["fastest_lap_seconds"] = df["fastest_lap"].apply(lap_to_seconds)
df.to_csv("output/f1_2025_cleaned.csv", index=False)
```

5.4 Data Quality After Cleaning

- The cleaned dataset has shape **479 rows × 25 columns** with the correct data types.
- All missing values in the analysis columns are resolved. The engineered fastest_lap_seconds column intentionally keeps 22 empty values for drivers who never set a timed lap.
- Country labels are now consistent (single 'United Kingdom' value), dates are true datetimes and every numeric gap is non-negative.
- The cleaned file output/f1_2025_cleaned.csv is the single source used by all later analysis and charts.

6. Data Analysis with NumPy

Every numeric column was converted into a NumPy array with `to_numpy()` and summarised using NumPy statistical functions. The table below reports the six required measures — average, minimum, maximum, total, standard deviation — plus the median.

Variable	Count	Average	Minimum	Maximum	Total	Std. Dev.	Median
points	479	5.061	0.00	25.00	2424.0	7.176	1.00
gap_to_winner_seconds	479	32.542	0.00	99.723	15587.7	28.917	25.532
grid_position	479	10.480	1.00	20.00	5020.0	5.756	10.00
finish_position	479	10.480	1.00	20.00	5020.0	5.756	10.00
laps_completed	479	55.182	0.00	78.00	26432.0	16.611	57.00
fastest_lap_seconds	457	87.021	67.924	136.121	39768.4	10.428	86.765

Key reading: a total of **2,424 championship points** were awarded across the season. The mean points per entry is only 5.06 with a high standard deviation (7.18), showing that points are heavily concentrated among the front-runners while roughly half of all entries score nothing (median = 1).

6.1 NumPy Code Used — VSEC Syllabus Concepts

```
# Numerical computing with NumPy arrays
points = clean_df["points"].to_numpy(dtype=float)
np.size(points) # count -> 479
np.mean(points) # average -> 5.061
np.median(points) # median -> 1.0
np.std(points) # std dev -> 7.176
np.min(points) # minimum -> 0.0
np.max(points) # maximum -> 25.0
np.sum(points) # total -> 2424.0

# Correlation matrix of seven numeric variables
corr = np.corrcoef(clean_df[corr_cols].dropna().to_numpy(),
rowvar=False)
# Championship aggregation with NumPy sum
standings = clean_df.groupby("driver")["points"] \
.apply(lambda s: np.sum(s.to_numpy()))
```

6.2 Correlation Analysis (np.corrcoef)

A Pearson correlation matrix was computed with NumPy's `np.corrcoef()` on seven numeric variables. The strongest relationships are summarised below; the full matrix is visualised as a heatmap in Section 7.

Variable Pair	Correlation	Interpretation
finish_position & points	–0.86	Very strong inverse link — the higher you finish, the more points you score.
grid_position & points	–0.72	Strong inverse link — qualifying well strongly supports a points finish.
grid_position & finish_position	+0.69	Strong positive link — starting position largely determines the result.
laps_completed & fastest_lap_seconds	–0.59	Longer, slower circuits show larger lap times; more laps tend to mean shorter layouts.
grid_position & grid_places_gained	+0.44	Drivers starting further back have more opportunity to gain places.

6.3 Driver Championship Standings

Rank	Driver	Total Points
1	Lando Norris	394
2	Max Verstappen	389
3	Oscar Piastri	381
4	George Russell	289
5	Charles Leclerc	225
6	Lewis Hamilton	135
7	Andrea Kimi Antonelli	135
8	Alexander Albon	70
9	Carlos Sainz	54
10	Fernando Alonso	51
11	Nico Hülkenberg	51
12	Isack Hadjar	50

Lando Norris (McLaren) takes the 2025 World Drivers' Championship with **394 points**, narrowly ahead of Max Verstappen (389) and Oscar Piastri (381). Note that two drivers appear under two teams because of mid-season driver swaps; their points are correctly combined above.

6.4 Constructor Championship Standings

Rank	Constructor Team	Total Points
1	McLaren	775

Rank	Constructor Team	Total Points
2	Mercedes	424
3	Red Bull Racing	410
4	Ferrari	360
5	Williams	124
6	Racing Bulls	88
7	Aston Martin	80
8	Haas F1 Team	73
9	Kick Sauber	70
10	Alpine F1 Team	20

6.5 Season Honours Tally

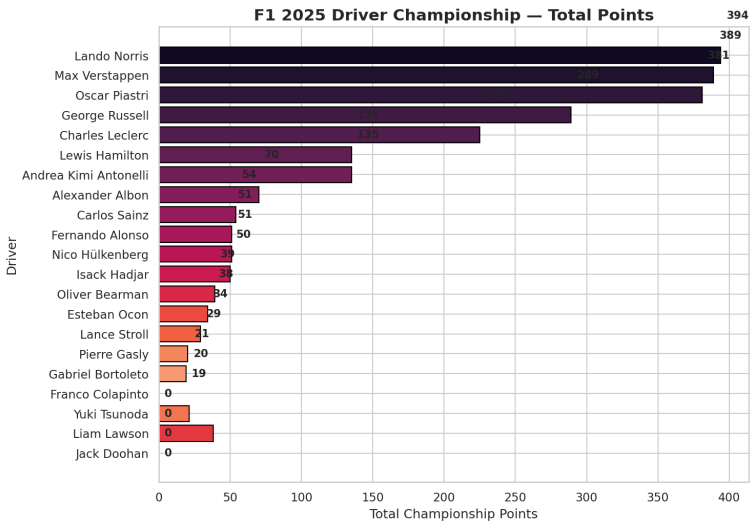
Driver	Wins	Podiums	Poles	Fastest Laps
Max Verstappen	8	15	8	3
Lando Norris	7	18	7	6
Oscar Piastri	7	16	6	6
George Russell	2	9	2	3
Carlos Sainz	0	2	0	0
Alexander Albon	0	0	0	1
Andrea Kimi Antonelli	0	3	0	3
Isack Hadjar	0	1	0	0
Charles Leclerc	0	7	1	1
Lewis Hamilton	0	0	0	1
Nico Hülkenberg	0	1	0	0

6.6 Additional NumPy Findings

- Race-result reliability: 330 Finished (68.9 %), 89 Lapped (18.6 %), 51 Retired (10.7 %), 6 Disqualified (1.3 %) and 3 Did Not Start (0.6 %).
- Best average finishing position among title contenders: Verstappen 3.67, Piastri 4.21, Norris 4.25, Russell 4.54.
- Sprint weekends averaged 5.050 points per entry versus 5.064 on normal weekends — effectively identical scoring intensity.
- The fastest lap recorded all season was 67.924 seconds and the slowest was 136.121 seconds, reflecting the wide variety of circuit layouts.

7. Data Visualization with Matplotlib / Seaborn

Ten charts were generated with proper titles, axis labels, legends and colour schemes. Each chart is shown alongside the actual code concept used to build it (mapped to the VSEC syllabus), followed by a short interpretation.



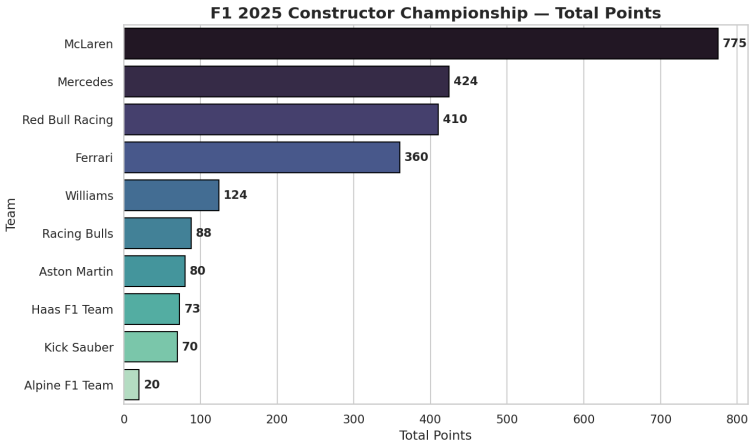
VSEC SYLLABUS CONCEPT

Matplotlib — Horizontal Bar Chart (ax.barh)

```
fig, ax =
plt.subplots(figsize=(10, 7))
d =
driver_pts.sort_values("points")
ax.barh(d["driver"], d["points"],
color=colors, edgecolor="black")
for y, v in
enumerate(d["points"]):
ax.text(v + 4, y, str(v),
va="center", fontweight="bold")
ax.set_title("Driver Championship
— Points")
ax.set_xlabel("Total Championship
Points")
```

Figure 7.1 — Horizontal bar chart of total championship points per driver

The bar chart ranks all 21 drivers by total points. The three title contenders — Norris (394), Verstappen (389) and Piastri (381) — are separated by only 13 points, while a clear gap separates them from Russell in fourth place (289).



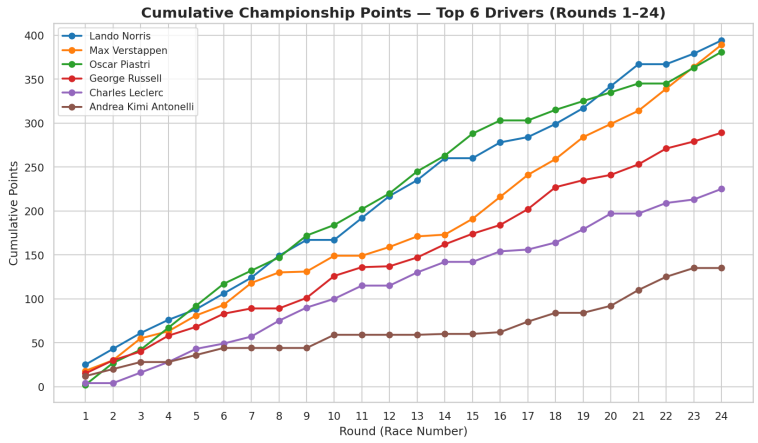
VSEC SYLLABUS CONCEPT

Seaborn — Categorical Bar Plot (sns.barplot)

```
sns.barplot(data=team_pts,
x="points",
y="team", hue="team",
legend=False,
palette="mako", edgecolor="black",
ax=ax)
ax.set_title("Constructor
Championship")
ax.set_xlabel("Total Points")
ax.set_ylabel("Team")
```

Figure 7.2 — Constructor championship points per team

McLaren dominates with 775 points — almost double the next best team (Mercedes, 424). Red Bull (410) and Ferrari (360) complete the top four, while Alpine (20) lies far behind, illustrating the highly unequal competitive order of the field.



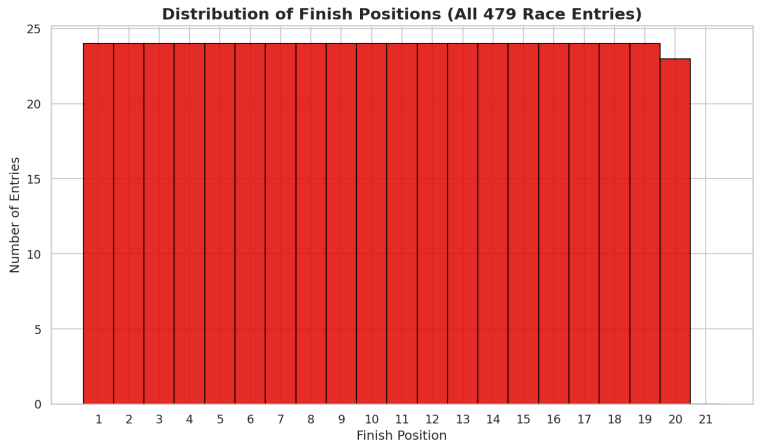
VSEC SYLLABUS CONCEPT

Matplotlib — Line Chart (ax.plot) with NumPy cumsum

```
for drv in top6:
sub = clean_df[clean_df["driver"]
== drv]
sub = sub.sort_values("round")
cumulative = np.cumsum(sub["points
"].to_numpy())
ax.plot(sub["round"], cumulative,
marker="o", linewidth=2,
label=drv)
ax.set_xticks(np.arange(1, 25))
ax.legend(loc="upper left")
```

Figure 7.3 — Line chart of cumulative points for the top six drivers

The line chart traces how the championship battle evolved round by round. The McLaren pair and Verstappen run closely together all season, repeatedly exchanging the lead, while Russell and Leclerc form a steady second group and the remaining drivers plateau.



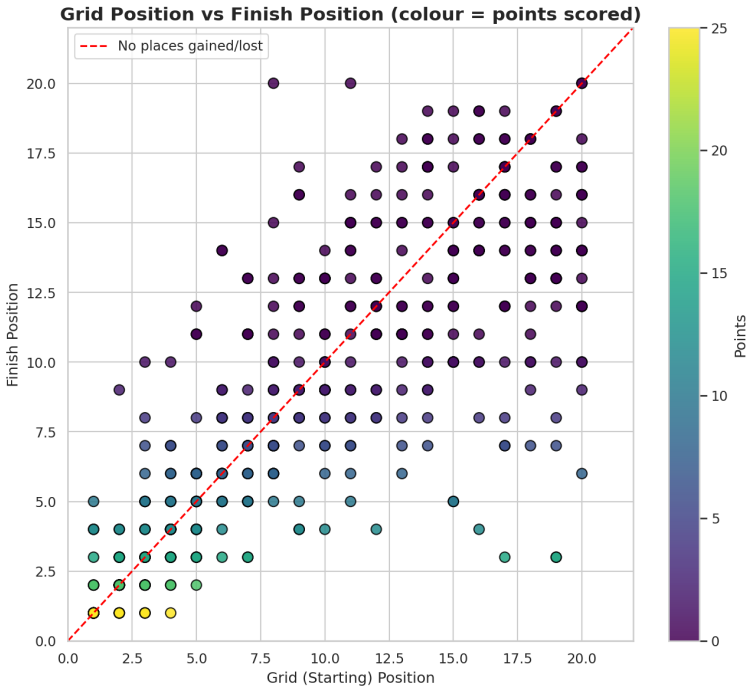
VSEC SYLLABUS CONCEPT

Matplotlib — Histogram (ax.hist) with NumPy bins

```
ax.hist(clean_df["finish_position"
],
bins=np.arange(0.5, 22.5, 1),
color="#e10600",
edgecolor="black",
alpha=0.85)
ax.set_title("Distribution of
Finish Positions")
ax.set_xlabel("Finish Position")
ax.set_ylabel("Number of Entries")
```

Figure 7.4 — Histogram of finish positions across all 479 entries

The histogram is roughly uniform for positions 1–20 but with taller bars at the front and back. Every position is occupied roughly 20–25 times (once per race), confirming a complete, well-populated grid with no missing finishing slots.



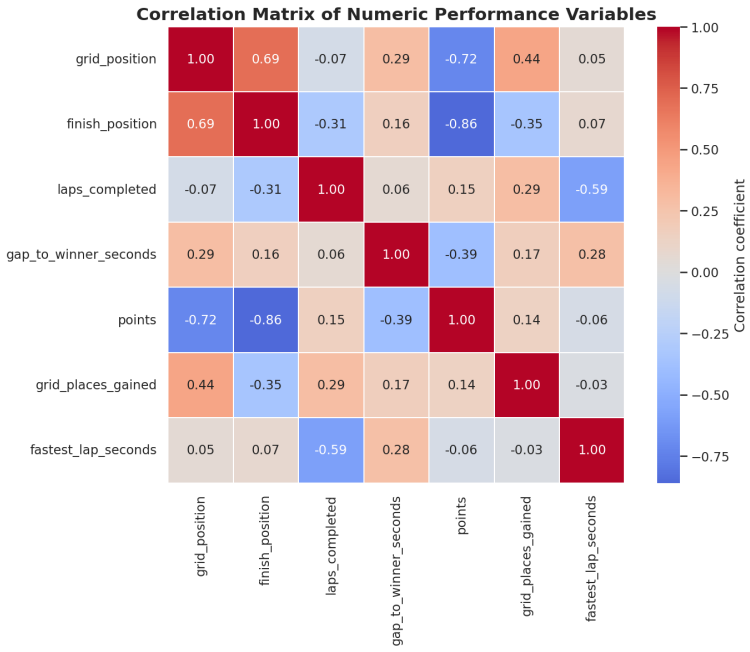
VSEC SYLLABUS CONCEPT

Matplotlib — Scatter Plot (ax.scatter) + colour bar

```
sc = ax.scatter(classified["grid_position"],
classified["finish_position"],
c=classified["points"],
cmap="viridis", s=70,
edgecolor="black")
ax.plot([0, 22], [0, 22], ls="--",
color="red", label="No places gained/lost")
fig.colorbar(sc, ax=ax,
label="Points")
```

Figure 7.5 — Scatter plot of grid position vs finish position (colour = points)

Points cluster below the red diagonal line for front-row starters, meaning they usually finish where or better than they qualified. Bright yellow points (25 points, race wins) appear almost exclusively in the bottom-left corner — pole and front-row starts. This visually confirms the strong grid–finish relationship.



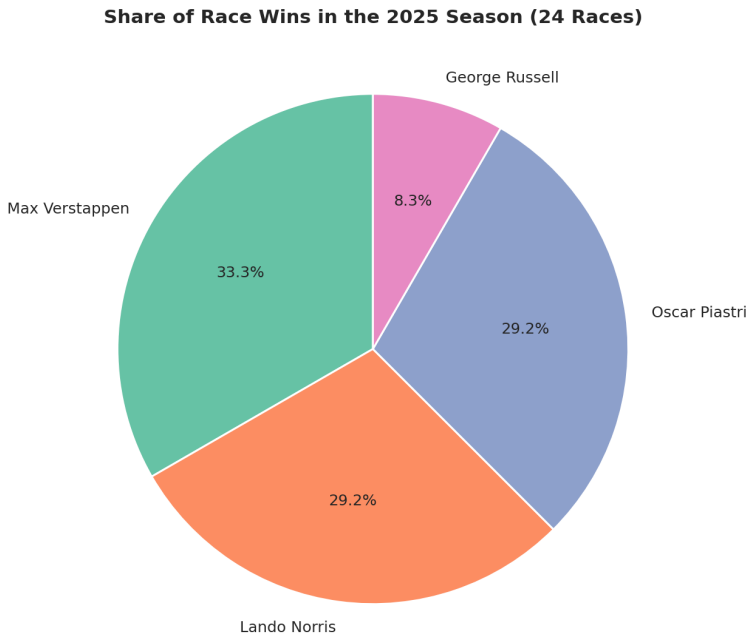
VSEC SYLLABUS CONCEPT

Seaborn — Correlation Heatmap (sns.heatmap)

```
sns.heatmap(corr_df, annot=True,
cmap="coolwarm", center=0,
fmt=".2f", linewidths=0.8,
square=True, ax=ax,
cbar_kws={"label":
"Correlation coefficient"})
ax.set_title("Correlation Matrix")
```

Figure 7.6 — Seaborn heatmap of the NumPy correlation matrix

Deep red cells show strong positive correlations and deep blue cells strong negative ones. The –0.86 cell between finish position and points is the dominant relationship, and the –0.72 grid-to-points cell reinforces how valuable qualifying performance is.



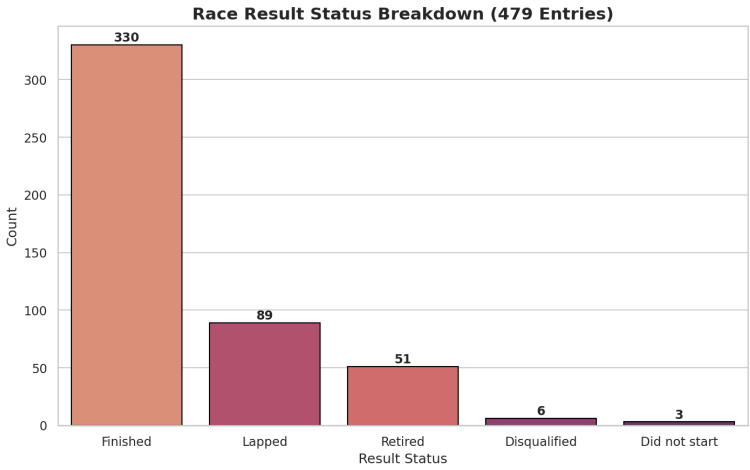
VSEC SYLLABUS CONCEPT

Matplotlib — Pie Chart (ax.pie)

```
ax.pie(win_data["wins"],
labels=win_data["driver"],
autopct="%1.1f%%", startangle=90,
colors=sns.color_palette("Set2"),
wedgeprops={"edgecolor": "white",
"linewidth": 1.5})
ax.set_title("Share of Race Wins
(24 Races)")
```

Figure 7.7 — Pie chart showing the share of race wins

Only four drivers won a race in 2025. Verstappen holds the largest share with 8 wins (33.3 %), followed by Norris and Piastri on 7 each (29.2 % each) and Russell with 2 (8.3 %). The two McLaren drivers together account for 14 of the 24 wins.



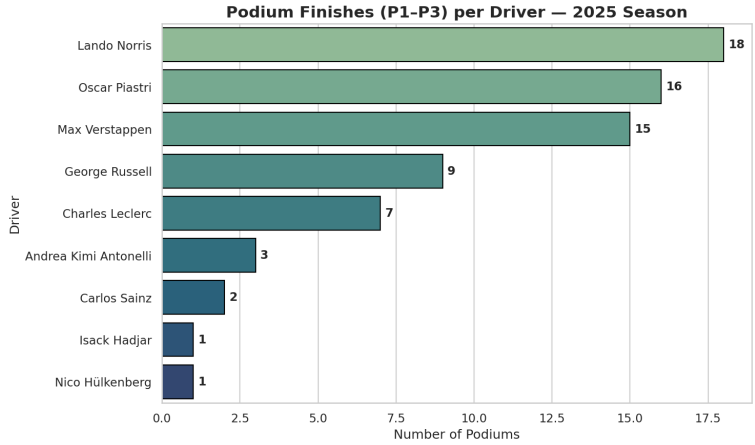
VSEC SYLLABUS CONCEPT

Seaborn — Count Plot (sns.countplot)

```
order = (clean_df["result_status"]
.value_counts().index)
sns.countplot(data=clean_df,
x="result_status", order=order,
hue="result_status", legend=False,
palette="flare",
edgecolor="black")
ax.set_title("Race Result Status
Breakdown")
```

Figure 7.8 — Seaborn count plot of race-result status

Finished is by far the most common outcome (330), followed by Lapped (89). The 51 Retired entries highlight that mechanical reliability and race incidents still end roughly one in ten weekends early, while disqualifications and DNS events are rare.



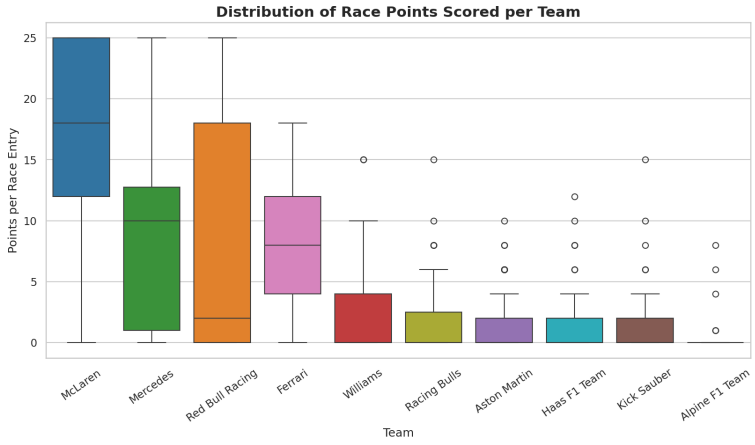
VSEC SYLLABUS CONCEPT

Seaborn — Horizontal Bar Plot (sns.barplot)

```
sns.barplot(data=pod, x="podiums",
y="driver", hue="driver",
legend=False, palette="crest",
edgecolor="black", ax=ax)
for i, v in
enumerate(pod["podiums"]):
ax.text(v + 0.15, i, str(v),
va="center", fontweight="bold")
```

Figure 7.9 — Bar chart of podium finishes (P1–P3) per driver

Norris leads the podium count with 18, meaning he finished on the rostrum in 75 % of the races. Piastri (16) and Verstappen (15) are close behind, and only seven drivers appeared on the podium at all — evidence of how concentrated front-running performance is in modern Formula 1.



VSEC SYLLABUS CONCEPT

Seaborn — Box Plot (sns.boxplot) for distributions

```
sns.boxplot(data=clean_df,
x="team",
y="points", order=team_order,
hue="team", legend=False,
palette="tab10", ax=ax)
ax.set_title("Points Distribution
per Team")
ax.tick_params(axis="x",
rotation=35)
ax.set_ylabel("Points per Race
Entry")
```

Figure 7.10 — Box plot of the points distribution per race entry, by team

The box plot shows both the median and the spread of each team's race-by-race points. McLaren's box sits highest with a wide, high-scoring range, while Alpine's narrow box near zero shows almost no points-paying finishes — a direct measure of consistency.

8. Findings / Observations

The cleaning, numerical analysis and visualisations together reveal the following concrete observations about the 2025 Formula 1 season:

1. **Lando Norris is the 2025 World Drivers' Champion** with 394 points, but the title battle was extremely tight — just 13 points covered the top three drivers (Norris 394, Verstappen 389, Piastri 381).
2. **McLaren won the Constructors' Championship decisively** with 775 points, nearly doubling Mercedes (424); its two drivers scored 14 of the 24 race wins between them.
3. **Only four drivers won a race all season** — Verstappen (8), Norris (7), Piastri (7) and Russell (2) — showing that race victories are highly concentrated among a small elite group.
4. **Qualifying strongly determines the race outcome:** grid position and finish position correlate at +0.69, and grid position and points at –0.72. Starting at the front is a massive advantage.
5. **The strongest statistical relationship is finish position vs points (–0.86)**, confirming that the F1 points-scoring system rewards finishing position almost perfectly.
6. **Reliability is generally high:** 87.5 % of entries were classified (Finished or Lapped), while 10.7 % Retired; roughly one in ten race weekends still ends early.
7. **Points are heavily skewed:** the median entry scores only 1 point even though the mean is 5.06, because a small number of front-runners capture most of the 2,424 points available.
8. **Data cleaning was essential:** 181 missing cells, inconsistent 'UK' labels and 5 invalid negative gap values had to be corrected before any reliable analysis was possible.

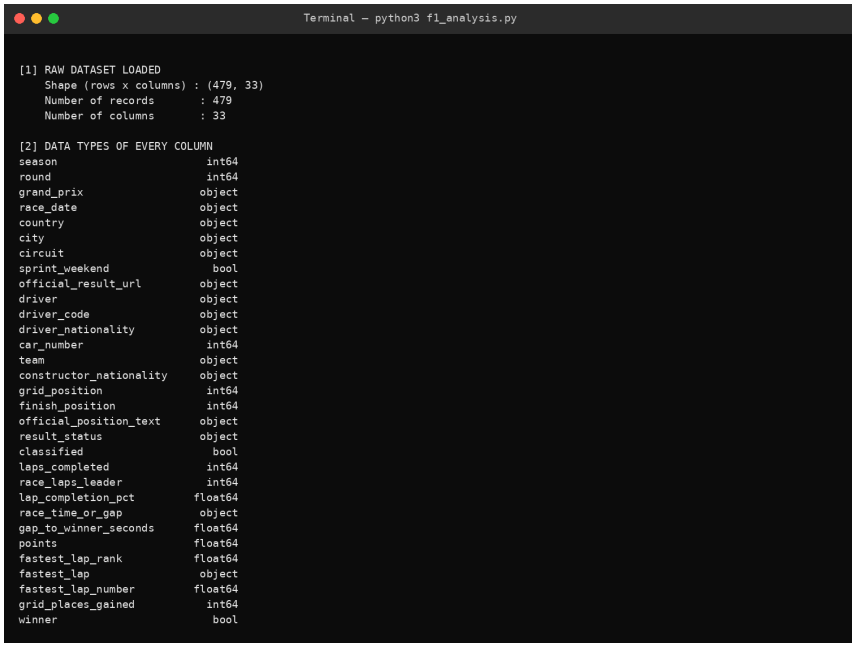
9. Result / Screenshots

This section presents screenshots of the actual project in action — the Python source code in the editor and the real output produced in the terminal. A brief description is written below every screenshot explaining the screen and its functionality.



Screenshot 9.1 — Data-cleaning code (Pandas) in the code editor

This screen shows the Part (a) Pandas code: loading the CSV, inspecting shape and missing values, converting dates, renaming the 'UK' label, filling missing gaps and correcting the invalid negative gap values. Syntax highlighting distinguishes keywords, strings and comments.



Screenshot 9.2 — Terminal output of the data-cleaning stage

The executed program prints the raw (479, 33) shape, every column's data type, the duplicate count (0) and the full missing-value report, then confirms each cleaning action and the final zero-missing cleaned dataset.

f1_analysis.py — Data Analysis (NumPy)

```
def numpy_summary(name, series):
    arr = series.to_numpy(dtype=float)
    return {
        "mean": np.mean(arr), "min": np.min(arr),
        "max": np.max(arr), "sum": np.sum(arr),
        "std": np.std(arr), "median": np.median(arr)}

points = clean_df["points"].to_numpy()
print(np.mean(points), np.std(points), np.sum(points))

# NumPy correlation matrix
cols = ["grid_position", "finish_position", "points",
        "laps_completed", "gap_to_winner_seconds"]
corr = np.corrcoef(clean_df[cols].dropna().to_numpy())

# championship standings with NumPy sum
standings = (clean_df.groupby("driver")["points"]
             .apply(lambda s: np.sum(s.to_numpy()))
             .sort_values(ascending=False))
```

Screenshot 9.3 — NumPy analysis code in the code editor

This screen shows the Part (b) code: a reusable NumPy summary function built from np.mean, np.min, np.max, np.sum, np.std and np.median, the np.corrcoef correlation matrix and the groupby standings that aggregate points with NumPy sums.

Terminal — python3 f1_analysis.py

```
[1] NUMERICAL SUMMARY STATISTICS (using NumPy functions)

  NumPy statistics for 'points':
    Count           : 479
    Average (mean)  : 5.061
    Minimum         : 0.000
    Maximum         : 25.000
    Total (sum)     : 2424.0
    Std. deviation  : 7.176
    Median          : 1.000

  NumPy statistics for 'gap_to_winner_seconds':
    Count           : 479
    Average (mean)  : 32.542
    Minimum         : 0.000
    Maximum         : 99.723
    Total (sum)     : 15587.7
    Std. deviation  : 28.917
    Median          : 25.532

  NumPy statistics for 'grid_position':
    Count           : 479
    Average (mean)  : 10.480
    Minimum         : 1.000
    Maximum         : 20.000
    Total (sum)     : 5020.0
    Std. deviation  : 5.756
    Median          : 10.000

  NumPy statistics for 'finish_position':
    Count           : 479
    Average (mean)  : 10.480
    Minimum         : 1.000
    Maximum         : 20.000
    Total (sum)     : 5020.0
    Std. deviation  : 5.756
    Median          : 10.000
```

Screenshot 9.4 — Terminal output of the NumPy statistics

The program prints the count, average, minimum, maximum, total, standard deviation and median for points and the other key numeric variables, exactly as required by the analysis section of the assignment.

f1_analysis.py — Visualization (Matplotlib/Seaborn)

```python
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")

# 1. bar chart - driver championship points
fig, ax = plt.subplots(figsize=(10, 7))
ax.barh(driver_pts["driver"], driver_pts["points"])
ax.set_title("F1 2025 Driver Championship - Total Points")
ax.set_xlabel("Total Championship Points")

# 2. heatmap - correlation matrix
sns.heatmap(corr_df, annot=True, cmap="coolwarm", center=0

# 3. scatter - grid vs finish position
ax.scatter(df["grid_position"], df["finish_position"], c=df["points"])

# 4. histogram - finish position distribution
plt.hist(clean_df["finish_position"], bins=range(1, 22))
plt.savefig("charts/04_finish_position_histogram.png")
```

Screenshot 9.5 — Visualisation code (Matplotlib / Seaborn)

This screen shows the Part (c) charting code — a horizontal bar chart with Matplotlib, a Seaborn correlation heatmap, a grid-vs-finish scatter plot and a histogram, each with explicit titles and axis labels.

Terminal — python3 f1_analysis.py

```
# PART (c) — DATA VISUALIZATION WITH MATPLOTLIB / SEABORN
################################################################################

[1/10] Driver Championship standings — Bar chart
    Saved chart -> /home/user/f1_project/charts/01_driver_standings.png
[2/10] Constructor Championship standings — Bar chart
    Saved chart -> /home/user/f1_project/charts/02_team_standings.png
[3/10] Points progression across all 24 rounds — Line chart
    Saved chart -> /home/user/f1_project/charts/03_points_progression.png
[4/10] Finish position distribution — Histogram
    Saved chart -> /home/user/f1_project/charts/04_finish_position_histogram.png
[5/10] Grid position vs Finish position — Scatter plot
    Saved chart -> /home/user/f1_project/charts/05_grid_vs_finish_scatter.png
[6/10] Correlation matrix — Heatmap
    Saved chart -> /home/user/f1_project/charts/06_correlation_heatmap.png
[7/10] Race wins share — Pie chart
    Saved chart -> /home/user/f1_project/charts/07_wins_pie.png
[8/10] Race result status breakdown — Count plot
    Saved chart -> /home/user/f1_project/charts/08_status_countplot.png
[9/10] Podium finishes per driver — Bar chart
    Saved chart -> /home/user/f1_project/charts/09_podiums_bar.png
[10/10] Points distribution per team — Box plot
    Saved chart -> /home/user/f1_project/charts/10_team_points_boxplot.png

================================================================================
 ANALYSIS COMPLETE SUCCESSFULLY
 Cleaned data : output/f1_2025_cleaned.csv
 Charts       : charts/  (10 PNG files)
 Tables       : output/  (standings, honours, correlation)
 GitHub repos :
   Soham Hande   : https://github.com/handesoham1234-tech/Soham_Hande_19
   Anshu Navghare: https://github.com/maverick8307/anshu_02
================================================================================
```

Screenshot 9.6 — Terminal output confirming all ten charts were saved

The final part of the run logs each chart being written to the charts/ folder and ends with the 'ANALYSIS COMPLETE SUCCESSFULLY' banner, listing the cleaned data, charts, tables and both team GitHub repositories.

10. GitHub Repository Links

The complete source code, the raw and cleaned datasets, all ten charts, the screenshots and this report are uploaded to **both team members' public GitHub repositories**. Either repository can be cloned or browsed directly using the links given below.

Soham Hande	Anshu Navghare
github.com/handesoham1234-tech/Soham_Hande_19	github.com/maverick8307/anshu_02

Soham Hande

github.com/handesoham1234-tech/Soham_Hande_19

Anshu Navghare

github.com/maverick8307/anshu_02

Scan either QR code with a mobile phone to open that repository instantly

10.1 Contents of Each Repository

- **f1_analysis.py** — the complete, well-commented Python program (cleaning + NumPy analysis + 10 charts).
- **F1_Analysis_Notebook.ipynb** — runnable Jupyter / Google Colab notebook version with outputs.
- **f1_2025_grand_prix_results.csv** — the original raw dataset (479 records, 33 columns).
- **output/** — cleaned dataset, driver & constructor standings, honours, correlation matrix and console log.
- **charts/** — all ten high-resolution visualisations used in Section 7.
- **screenshots/** — IDE and terminal screenshots and the two GitHub QR codes.
- **F1_2025_Mini_Project_Report.pdf** — this project report.

To clone the repositories locally, run:

```
git clone https://github.com/handesoham1234-tech/Soham_Hande_19.git
git clone https://github.com/maverick8307/anshu_02.git
```

11. Conclusion

The project demonstrated how Python libraries can be used to clean, analyse and visualise a real-world sports dataset. Pandas was used for data cleaning, NumPy for numerical analysis, and Matplotlib and Seaborn for visualisation. The analysis helped identify useful patterns and trends in the Formula 1 2025 season.

Starting from a raw 479-record file containing 181 missing cells, inconsistent country labels and invalid negative values, the team produced a clean 25-column dataset and used it to answer the core sporting questions of the season: who won the championships, how dominant each team was, how strongly qualifying influences the race result, and how reliable the cars proved to be. Ten different charts turned these numbers into clear, interpretable pictures.

11.1 Skills Gained

- Hands-on experience of the complete data-science pipeline: ingest → clean → analyse → visualise → report.
- Practical use of Pandas for missing-value handling, type conversion, feature engineering and grouping.
- Confidence with NumPy array statistics and correlation analysis.
- Ability to choose the correct chart type and build publication-quality Matplotlib/Seaborn graphics.
- Experience packaging reproducible code on GitHub with documentation and version control.

11.2 Future Enhancements

- Extend the study to multiple seasons (2020–2025) to analyse long-term team and driver trends.
- Add predictive modelling with scikit-learn to forecast race results from qualifying data.
- Build an interactive dashboard using Plotly / Streamlit for filtering by team, driver and circuit.
- Include qualifying and sprint-race datasets for a deeper analysis of weekend performance.

12. References

[1] Official Formula 1 Results & Statistics — Formula 1 World Championship, https://www.formula1.com/en/results.html

[2] Pandas Development Team, *Pandas Documentation — Python Data Analysis Library*, https://pandas.pydata.org/docs/

[3] NumPy Developers, *NumPy Reference Documentation*, https://numpy.org/doc/

[4] Matplotlib Development Team, *Matplotlib: Visualization with Python*, https://matplotlib.org/stable/contents.html

[5] Waskom, M. L., *Seaborn: Statistical Data Visualization*, https://seaborn.pydata.org/

[6] ReportLab, *ReportLab PDF Generation User Guide*, https://docs.reportlab.com/

[7] Python Software Foundation, *The Python Language Documentation*, https://docs.python.org/3/

[8] Soham Hande — Project GitHub Repository (source code, datasets, charts, report): https://github.com/handesoham1234-tech/Soham_Hande_19

[9] Anshu Navghare — Project GitHub Repository (source code, datasets, charts, report): https://github.com/maverick8307/anshu_02

[10] McKinney, W., *Python for Data Analysis*, 3rd Edition, O'Reilly Media, 2022.

[11] VanderPlas, J., *Python Data Science Handbook*, 2nd Edition, O'Reilly Media, 2022.

END OF PROJECT REPORT